

NVSwitch and ConnectX

The NVSwitch module is module ID 1. It discovers supported NVSwitch devices, NVSwitch links, and ConnectX adapters; collects their telemetry; and makes that information available through DCGM's common entity and field interfaces. The implementation retains its historical NVSwitch module name even though its current entity path also covers ConnectX. DCGM can load the module automatically when supported hardware is present or when an operation needs it.

When this module is useful

NVSwitch connects GPUs within systems whose fabric is larger than a direct GPU-to-GPU NVLink topology. An operator normally uses this module to answer questions such as:

- Did DCGM discover every switch expected in this server?
- Are all expected GPU and switch links up before a job starts?
- Did a switch report a fatal or non-fatal error while a job was running?
- Is traffic crossing a particular switch or switch port?
- Is a communication failure local to one GPU link, one switch port, or the fabric more broadly?
- Is a ConnectX adapter healthy and operating at its expected PCIe width and speed?

These checks fit naturally into node qualification and workload triage. Check the link topology before admitting a communication-heavy job, leave passive health watches enabled during the job, and inspect per-link telemetry when a collective workload fails or performs unevenly.

On DGX-2 and HGX-2 systems, run `nv-hostengine` as root for the NVSwitch module. [NVIDIA Fabric Manager](#) is a separate service responsible for configuring the NVSwitch fabric and must be installed and running. DCGM observes the fabric; it does not configure the fabric or manage the Fabric Manager lifecycle. A healthy DCGM report also does not prove that a distributed communication library or workload is configured correctly.

Entities and fields

DCGM represents each switch as a `DCGM_FE_SWITCH` entity. The command-line selector is `nvswitch:<switch-id>`. Each switch port is a `DCGM_FE_LINK` entity and can be selected with `switch_link:<switch-id>:<link-index>`. GPU NVLinks and NVSwitch links share parts of the monitoring interface, but they remain distinct entities.

A discovered ConnectX adapter is a `DCGM_FE_CONNECTX` entity and uses the selector `cx:<id>`. Its fields cover health, active and expected PCIe link width and speed, correctable and uncorrectable PCIe error state, and device temperature. ConnectX is not an NVLink port, so `dcgmi nvlink` does not show these adapter fields.

Discover IDs at runtime. Switch IDs and link IDs are host-engine inventory, not stable identifiers to copy between systems. The most useful commands are:

- `dcgmi discovery --list` for switch inventory.
- `dcgmi nvlink --link-status --show-entity-ids` for GPU and switch-port state and link identifiers.
- `dcgmi dmon --list` for the field catalogue in the installed DCGM release.

If discovery lists no NVSwitches, switch-specific examples on this page do not apply to that host. A system can still have direct GPU NVLinks or a supported ConnectX adapter without an NVSwitch.

Example: inspect a fabric problem

Start by inventorying the host and checking every link. The status key distinguishes links that are up, down, disabled, or unsupported.

```
$ dcgmi discovery --list
$ dcgmi nvlink --link-status --show-entity-ids
```

Suppose the output identifies switch 0. The following command samples its temperature and aggregate transmit and receive throughput five times. Field 858 is current switch temperature; fields 861 and 862 are aggregate transmit and receive throughput.

```
$ dcgmi dmon --entity-id nvswitch:0 --field-id 858,861,862 --count 5
```

If the link-state report points to port 3, narrow the observation to that port. Fields 780 and 781 are per-link transmit and receive throughput; fields 782 and 783 report fatal and non-fatal link errors.

```
$ dcgmi dmon --entity-id switch_link:0:3 \
  --field-id 780,781,782,783 --count 5
```

An unsupported field, an entity/field mismatch, or a value that has not yet been sampled can appear as `N/A`. Confirm field names and scope with `dcgmi dmon --list` and the field reference before interpreting `N/A` as a hardware failure. Link counters are most useful when compared over the same workload interval rather than as isolated values.

Example: inspect a ConnectX adapter

`dcgmi discovery --list` reports the runtime ConnectX IDs. Suppose it lists adapter 0. Monitor its health, active and expected PCIe link properties, and temperature with fields 1300 through 1304 and 1310:

```
$ dcgmi dmon --entity-id cx:0 \  
--field-id 1300,1301,1302,1303,1304,1310 --count 5
```

An active width or speed below the corresponding expected value is useful evidence for a PCIe-path investigation. Interpret the health and error fields with the field reference rather than treating every nonzero bit pattern as the same incident.

For a persistent health watch, put the discovered adapters in a group and enable the ConnectX selector `x`:

```
$ dcgmi group --create fabric-adapters --add cx:0  
$ dcgmi health --set x --group <group-id>  
$ dcgmi health --check --group <group-id>
```

Passive health monitoring

For continuous monitoring, the health module can watch NVSwitch fatal and non-fatal conditions and down links. The built-in `s` group selects all NVSwitches. The `a` watch selector includes the NVSwitch checks as well as the other health systems supported for the selected entities.

```
$ dcgmi health --set a --group s  
$ dcgmi health --check --group s
```

Health watches consume telemetry over time, so a check immediately after enabling them may not yet have enough samples. See [Health Monitoring](#) for watch lifetime, result severity, and the distinction between passive health checks and active diagnostics.

Further reading

- [dcgmi discovery](#) documents NVSwitch and ConnectX inventory output and the source of each entity ID.
- [dcgmi nvlink](#) documents link states, counters, and link selectors.
- [dcgmi dmon](#) documents field selection, output, and `N/A` behavior.
- [dcgmi health](#) is the exhaustive passive health-watch command reference.
- [Field Identifiers](#), [Field Scope](#), and [Field APIs](#) describe the programmatic field interface.
- [Entities and Groups](#) explains how GPU, NVSwitch, adapter, and link entities are identified and selected.
- [Topology and NVLink](#) explains GPU relationships, current GPU and NVSwitch port state, concise error reports, and repeated link telemetry.



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2018-2026, NVIDIA Corporation.

Last updated on 2026-09-10.